DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2026

Input/Output in C

- Introduction
- Standard Stream I/O
- String I/O
- File I/O

Preliminaries

- Use `#include <stdio.h>` to get the declarations for the I/O functionality in the standard library.
- There are two different modes, text and binary.
- A text stream is a sequence of characters, composed of lines each ending with `'\n'`, also on macOS and Windows. The library will take care of line ending conversions when needed.
- A binary stream is a sequence of bytes.
- Streams are usually buffered.
- There are different functions for formatted and unformatted input/output.

`https://en.cppreference.com/w/c/io`

- Introduction
- Standard Stream I/O
- String I/O
- File I/O

Unformatted I/O to Standard Streams

`int putchar(int ch)`

- Writes `ch` to stdout, after conversion to `unsigned char`.
- Returns the printed character, or `EOF` on error.

`int getchar()`

- Reads and returns the next character from stdin.
- Returns `EOF` on error.

`EOF` is a macro for an implementation-defined negative `int`.

Example

(Remember: `char`s are just integers we like to visualise in a special way)

```
int main(void) {
    char c;
    while((c = getchar()) != EOF) {
        if(c >= 'A' && c <= 'Z')
            c = c - 'A' + 'a';
        putchar(c);
    }
    return 0;
}
```

- `./a.out < file.txt`, redirect stdin in the terminal.
- `cat file.txt | ./a.out`, pipe input from another program, in the terminal.

Formatted Output

```
int printf(const char *restrict format, ...)
```

- Writes formatted strings to stdout.
- The `...` is part of the signature (more later), it means a variadic number of arguments can be given.
- Also later: `const` and `restrict`.
- The number of characters printed is returned.
- The format string may contain literal text and conversion specifiers (they start with `%`).

```
printf("Hello World!\n"); // literal text
printf("%d\n", 10); // %d is for an integer
printf("Prices: %d and %d\n", 10, 20);
```

Format Strings

Each conversion specifier has the format:

1. The `%` character.
2. (Optional) Flags (e.g., justification and fill character)
3. (Optional) Width
4. (Optional) The `.` character followed by a precision.
5. (Optional) Length
6. Format specifier

`https://en.cppreference.com/w/c/io/fprintf`

Format Specifier

Specifier	Type*	Example	Output
d, i	int	`printf("%d", 10);`	10
u	unsigned int	`printf("%u", 10);`	10
o	unsigned int	`printf("%o", 10);`	12
x, X	unsigned int	`printf("%x", 10);`	a
f, F	double	`printf("%f", 2.3);`	2.300000
e, E	double	`printf("%e", 2.3);`	2.300000e+00
a, A	double	`printf("%a", 2.3);`	0x1.266666666666p+1
g, G	double	`printf("%g", 2.3);`	2.3
c	int	`printf("%c", 97);`	a
s	char*	`printf("%s", "Hi");`	Hi
p	void*	`printf("%p", p);`	0x564679f9b260
%	No argument	`printf("%%");`	%

The specifiers with a lower and upper case version differ in lower and upper case letter output.

* This is the default type. The length field changes this type.

https://en.cppreference.com/w/c/io/fprintf

Width and Flags

- The optional width parameter is the minimal length of the converted data.
- The flags for example specify justification and padding.

Example Format	Output (with _ here indicating a space)
`printf("%d", 10);`	`10`
`printf("%4d", 10);`	`__10`
`printf("%04d", 10);`	`0010`
`printf("%-4d", 10);`	`10__`
`printf("%+d", 10);`	`+10`
`printf("% d", 10);`	`_10`
`printf("% d", -10);`	`-10`
`printf("%s", "Hi");`	`Hi`
`printf("%4s", "Hi");`	`__Hi`

https://en.cppreference.com/w/c/io/fprintf

Precision

Example Format	Output
`printf("%.2f", 3.1415);`	`3.14`
`printf("%.0f", 3.1415);`	`3`
`printf("%.2e", 3.1415);`	`3.14e+00`
`printf("%.0e", 3.1415);`	`3e+00`
`printf("%.4s", "Hello");`	`Hell`
`printf("%.1s", "Hello");`	`H`

https://en.cppreference.com/w/c/io/fprintf

Formatted Input

```
int scanf(const char *restrict format, ...)
```

- The input version of **printf**.
- Read characters from stdin, according to the format string.
- A whitespace character in the format is used to read as many whitespace characters as possible in the input (no matter the type). See the **isspace** function.
- For each conversion specifier, a corresponding pointer must be given.
- Returns the number of arguments assigned, or **EOF**.

---

https://en.cppreference.com/w/c/io/fscanf

Examples

Assuming:

```
int i;
float f;
double d;
char *str;
```

Writing	Reading
`printf("%d", i);`	`scanf("%d", &i);`
`printf("%10d", i);`	`scanf("%10d", &i);`
`printf("%f", f);`	`scanf("%f", &f);`
`printf("%f", d);`	`scanf("%lf", &d);`
`printf("%s", str);`	`scanf("%s", str);`

Note: writing a `float`/`double` can be done with the same specifier `%f`, but reading is done with two different specifiers `%f` and `%lf`.

- Introduction
- Standard Stream I/O
- String I/O
- File I/O

I/O With Strings Instead of Files

```
int sprintf(char *restrict buffer, const char *restrict format, ...)
```

- Almost as **`printf`**, but output is written to the given **`buffer`**.
- No length check! Use **`snprintf`** if that is needed.

```
int sscanf(const char *restrict buffer, const char *restrict format, ...)
```

- Almost as **`scanf`**, but input is read from the given **`buffer`**.

- Introduction
- Standard Stream I/O
- String I/O
- File I/O

Opening a File

```
FILE *fopen(const char *restrict filename, const char *restrict mode);
```

- Mode: e.g., `r` (read only), `w` (write only), `a` (append).
- Default is text mode, append `b` to the mode for binary.
  E.g., `"rb"` to open a file in binary mode for reading.
- Returns a pointer to a file handle, or `NULL` on error.
- `FILE` is some implementation defined thing, just pass it around.
- `stdin`, `stdout`, `stderr` are macros for the special `FILE*`s for the standard streams.

---

https://en.cppreference.com/w/c/io/fopen

Closing a File

```
int fclose(FILE *stream);
```

- Closes the underlying file after flushing buffers.
- Automatically called on any open file at exit.

File Input

```
int getc(FILE *stream);
```

- As **getchar**, but for any file.

```
char *fgets(char *restrict str, int count, FILE *restrict stream);
```

- Read up to **count** characters, but stop at the end of file, or when the first line ending has been read.
- Stores data at the memory pointed to by **str**.
- Inserts a 0 in **str** after reading has stopped.
- Returns **str**, or **NULL** on error or end of file. Use **feof** to check if a file is at the end.

File Output

```
int putc(int c, FILE *stream);
```

- Writes a single character to the stream.

```
int fputs(const char *restrict str, FILE *restrict stream);
```

- Writes a string to the stream.

```
int fprintf(FILE *restrict stream, const char *restrict format, ...);
```

- As **printf** and **sprintf** but with an explicit file.

```
int fscanf(FILE *restrict stream, const char *restrict format, ... );
```

- As **scanf** and **sscanf** but with an explicit file.